

FinanceBench: Vector RAG vs. Vectorless RAG vs. Long Context

Master's thesis progress report · Philipps-Universität Marburg · Supervisor: Prof. Dr. Daniel Braun · 28 September 2026

1. Summary

All four architectures were run on the full FinanceBench open-source set (150 questions, 84 SEC filings) with the same generator (*gemini-3.1-flash-lite*), the same generation prompt and the same scoring rule. **Vectorless RAG is the strongest option**: it is the most accurate (tied with long context), the fastest, and it **dominates long context** (as accurate, 36% cheaper, ~8× faster). Vector RAG is the cheapest but the least accurate; the PageIndex hybrid sits in between. Across architectures, accuracy follows **how much relevant context reaches the generator**, up to about section size, not retrieval quality.

Architecture	Accuracy	Cost / 1k queries	Median latency	p95 latency	One-time indexing
Vectorless RAG (PageIndex)	79.3%	\$12.80	3.6 s	4.2 s	~\$15
Long context	78.7%	\$19.96	30.3 s	134.6 s	none
PageIndex hybrid	74.0%	\$4.54	4.6 s	28.8 s	~\$15 (reused)
Vector RAG (GAR)	70.7%	\$1.91	5.1 s	7.8 s	\$0 (local GPU)

2. What I built

Architecture	Retrieval	Generator sees (median)
Vector RAG (Kim et al., 2025)	Query expansion → Stella dense + BM25 ($\alpha = 0.85$, top 20) → Jina rerank (top 10) → LLM selection agent	1–2 chunks, ~0.8k tokens
Vectorless RAG (Lumer et al., 2025)	Per-document section tree (PageIndex); query expansion → LLM picks sections from the tree	full text of ~4 sections, ~33k tokens
PageIndex hybrid (own variant)	PageIndex navigation restricts the chunk pool → hybrid retrieval + rerank inside it; no selection agent	10 chunks, ~4.4k tokens
Long context	None; the whole PDF is sent inline with every question, no caching	whole filing, ~70k tokens

Held constant: generator, one structured generation prompt (preserve units, show calculations, JSON answer), judge (*gpt-oss-120b* via Groq, a different model family), dataset. Metrics: Correct / Incorrect / Failure to answer (Islam et al.); Recall@k and MRR at page level; cost = tokens × published price; latency from dedicated sequential timing passes (10 questions × 3 repeats; vectorless 5 × 3).

3. Problems found and how I handled them

Problem	Action
Free-tier limits (Gemini 3.5 Flash: 20 requests/day; Voyage rerank rate cap)	Used <i>gemini-3.1-flash-lite</i> for every pipeline; Jina rerank instead of Voyage.
PageIndex's classic tree builder failed on many filings (the LLM couldn't return the strict JSON table of contents)	Rebuilt all 84 trees with PageIndex Flash (structure from PDF layout; LLM only writes summaries).
Trees silently missing parts of documents (e.g. no headings before page 85 of 3M 2018 10-K)	Coverage check + audit; 16 failing documents (plus audit-flagged ones) rebuilt with my LLM table-of-contents fallback.
Navigation missed sections when question wording ≠ filing terminology	Query expansion before navigation: hit rate 92.7% → 94.0%.
Resumable stages can't give end-to-end latency	Separate sequential timing pass per pipeline; stages a pass skipped are added at their measured median.
Append-only cost logs held up to 14 duplicate records per question; indexing cost undercounted	De-duplicated by matching each record to the answer actually saved; fixed indexing clean-up to keep all calls of a build; 36 unlogged trees estimated at the logged average.
Scoring differed between pipelines: the number matcher ran on every question and its verdict was final for vector RAG / long context (it misreads e.g. "\$2,018mn" as 2.0)	One rule for all: matcher on calculation questions only, judge re-checks everything else. Re-scored vector RAG and long context; originals kept.
Generation prompts differed: vector RAG and long context used a short free-text prompt, vectorless and hybrid a structured one (show calculation, JSON answer)	Re-generated vector RAG and long context with the shared structured prompt (and re-timed them). Vector RAG 67.3% → 70.7%, long context 81.3% → 78.7%.

4. Follow-up experiments

All three below use the same prompt, model and scoring; p-values are paired McNemar tests vs. the control.

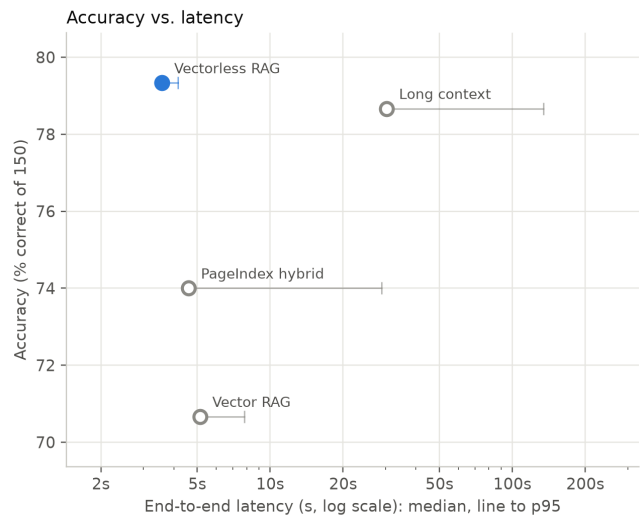
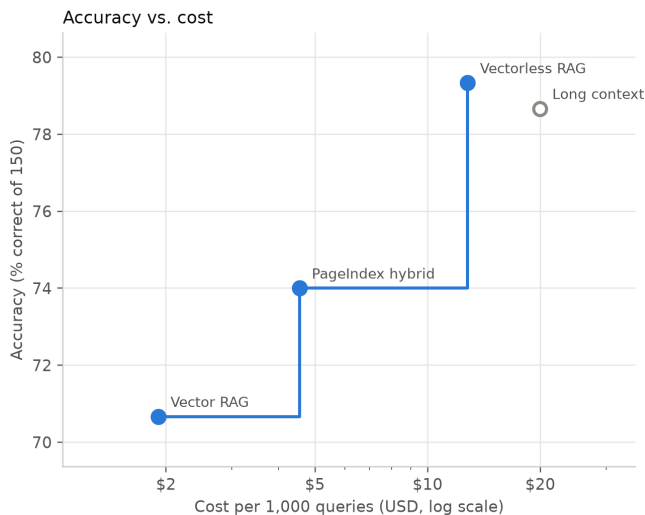
Variant	Tokens to generator	Accuracy	p vs control
Control: vector RAG's reranked top 10, no selection agent	4.5k	70.0%	—
Narrow to navigated pages first, then retrieve + rerank (hybrid)	4.4k	74.0%	0.29
Rerank top 10, then keep only chunks on navigated pages	3.1k (−31%)	66.0%	0.24

- **Selection agent:** keeps 1.6 of 10 chunks and discarded every gold-page chunk in 28 of 132 questions, yet removing it changes nothing: 70.0% without vs. 70.7% with ($p = 1.0$).
- **Narrowing first:** does not improve retrieval (gold page in top 10: 86% vs. 88% whole-document) and its accuracy gain is not significant.
- **Filtering after rerank:** saves tokens (~\$0.35 per 1k queries) but raises “Failure to answer” from 14 to 23; the gold page was removed in only 6 questions. PageIndex works better before retrieval than as a filter after it.

5. Results

	Calculation (50)	Domain-relevant (50)	Novel-generated (50)	All (150)
Vectorless RAG	94%	76%	68%	79.3%
Long context	94%	72%	70%	78.7%
PageIndex hybrid	88%	68%	66%	74.0%
Vector RAG	86%	66%	60%	70.7%

- **Significance (McNemar):** vectorless vs. long context $p = 1.0$ (tied). Vector RAG is significantly below vectorless ($p = 0.035$) and borderline against long context ($p = 0.05$); all other gaps are not significant.
- **Retrieval is not the bottleneck:** gold page in the reranked top 10 for 88% (vector RAG) and 86% (hybrid); vectorless navigation hits a gold section for 94%. 27 of vectorless's 31 non-correct answers had correct navigation.
- **Latency:** long context's time most likely goes to processing the PDF input (one filing took ~2.3 min median); vectorless reads ~33k tokens of text in about a second.



Pareto views (log x-axes). Filled points are on that view's frontier; hollow points are dominated in that view. Latency: median, line to p95.

6. Conclusions

- **RQ1 (accuracy):** vectorless 79.3% ≈ long context 78.7% > hybrid 74.0% > vector RAG 70.7% (only vectorless vs. vector RAG clearly significant).
- **RQ2 (retrieval, latency, cost):** retrieval recall is high for all retrieval pipelines; vectorless, hybrid and vector RAG answer in ~4–5 s, long context in 30 s; cost spans >10× (\$1.91–\$19.96 per 1k queries).
- **RQ3 (dominance):** no single winner, but vectorless dominates long context. Vectorless for accuracy and speed, vector RAG for cost, hybrid as a middle ground; long context only if zero preprocessing matters.
- **RQ4 (frontier):** vectorless → hybrid → vector RAG, trading accuracy for cost.

- **Key insight:** accuracy rises with context up to section level (0.8k → 4.4k → 33k tokens ↔ 70.7% → 74.0% → 79.3%), then stops: the whole filing (70k, 78.7%) adds nothing and is 8× slower. FinanceBench answers often need surrounding tables and notes, which small chunks miss.

7. Limitations and next steps

- One generator only; DeepSeek V4 not yet run (its API takes no PDF input, so long context needs a text path).
- Each architecture was generated once; 3 of 26 identical-input questions flipped across runs, so differences of a few questions are within noise. Repeated runs would tighten this.
- Latency samples are small (15–30 runs), so p95 is indicative only; borrowed-stage latencies are added as medians; vector RAG and long context were re-timed on a different day from vectorless and hybrid.
- Jina rerank is unpriced (\$0 counted); PageIndex indexing cost is partly estimated; vectorless tokens are estimated locally.
- Still to do: manual spot-check of 10–15% of judge verdicts.

Full analysis with live code: [analysis/findings_walkthrough.ipynb](#) (GitHub: [shaliqsv/financebench-rag-thesis](#)).